8장 병합과 충돌

깃은 언제든지 새로운 작업을 할 수 있는 브랜치를 만들 수 있습니다. 브랜치는 원본을 유지한 채로 새로운 기능을 개발하거나 버그를 수정하는 작업을 할 수 있는 기능입니다. 브랜치로 분기하여 코드를 수정했다면 언젠가는 원본에 다시 변경된 코드를 적용해야 합니다. 이 장에서는 파생된 브랜치 2개를 하나로 합치는 방법을 알아보겠습니다.

8.1 병합

8.2 Fast-Forward 병합

8.3 3-way 병합

8.4 브랜치 삭제

8.5 충돌

8.6 브랜치 병합 여부 확인

8.7 리베이스

8.8 정리

8.1 병합

브랜치를 생성하는 목적은 원본 코드에 영향을 주지 않고 분리하여 개발하기 위해서입니다. 독립된 브랜치에서 개발 작업이 끝나면 다시 원본 브랜치에 작업한 결과를 반영해야 합니다. 분리된 브랜치를 한 브랜치로 합치는 작업을 병합(합치기)이라고 합니다. 두 코드를 하나씩 직접 비교해 가며 수동으로 병합하거나 깃 같은 도구를 사용하여 자동으로 병합할 수 있습니다.

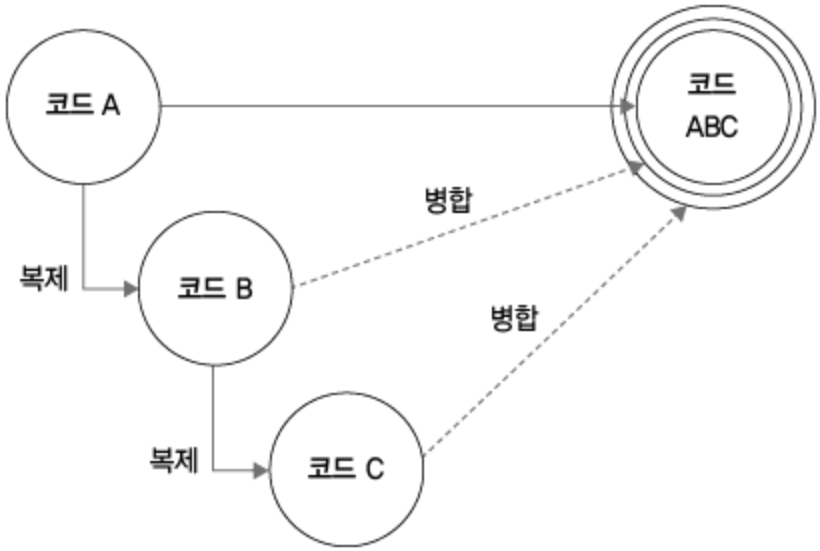
8.1.1 하나씩 직접 비교하는 수동 병합

소스 코드가 2개 있다고 합시다. 하나는 원본 소스 코드고, 또 하나는 수정된 소스 코드입니다. 수정된 코드는 버그를 고치고, 새로운 기능을 적용하여 테스트한 코드입니다. 개발을 끝내고 테스트를 완료했다면 수정 내역을 원본 소스 코드에 모두 반영해 주어야 합니다. 또는 수정된 코드 자체로 원본 코드를 대체할 수도 있습니다.

수동으로 병합하려면 양쪽 파일을 일일이 비교하며 바뀐 점을 찾아서 적용해야 합니다. 하지만 오류 없이 코드를 병합하는 것은 간단하지 않습니다. 몇 줄 안 되는 코드라면 몰라도 실제 개발할 때는 수천 줄 이상이기 때문에 두 코드를 하나로 합치는 것은 매우 힘든 작업입니다. 또 사람의 기억력은 한계가 있기 때문에 며칠 동안 작업한 내용을 일일이 옮겨 적는 것도 힘듭니다.

소스 코드가 2개보다 많다면 어떻게 될까요? 예를 들어 원본 소스 A를 복제하여 B를 수정하고, B를 복제하여 C도 수정하고 있습니다. 요즘은 협업 때문에 다수 개발자가 각각 새로운 기능들을 만듭니다. 또 365일 24시간 분업화된 상태에서 생긴 버그는 여러 개발자가 수정합니다. 이처럼 여러 개발자와 코드를 공유하면서 변경된 소스 코드를 병합하는 것은 매우 복잡합니다.

▼ 그림 8-1 코드 수동 병합



수정이 완료되면 B는 원본 소스 A에 수정 내역을 반영하는 작업을 해야 하고, C 또한 B를 반영하여 수정된 원본 소스 A에 또다시 수정 작업을 반영해야 합니다. 병합하려면 코드의 변경된 시점을 기억해야 합니다. 변경된 시점을 기준으로 순차적으로 병합 처리를 하는 것이 좀 더 쉽습니다. 변경된 시간 순서를 따르지 않고 병합하면 코드가 엉켜 잘못 동작할 확률이 높습니다.

8.1.2 깃으로 자동 병합

숙련된 개발자라고 해도 복잡한 수정 내역을 순차적인 시간에 따라 병합하는 것은 쉬운 일이 아닙니다. 깃을 사용하면 복잡한 파일을 좀 더 간편하게 병합할 수 있습니다.

Note

물론 깃이 모든 코드를 완벽하게 병합하는 것은 아닙니다. 아무리 좋은 도구라고 해도 자동으로 반영하지 못하는 것들이 있습니다. 이를 충돌이라고 합니다. 충돌은 8.5절에서 자세히 알아볼 것입니다.

깃의 자동 병합은 원본을 기준으로 두 파일의 변경 이력을 비교합니다. 변경된 파일 내용이 발견되면 자동으로 수정된 코드 내용을 병합합니다.

깃의 병합은 브랜치를 기준으로 합니다. 브랜치는 같은 저장소 내에서 서로 독립적으로 작업을 분리한 영역입니다. 분리된 각각의 브랜치에서 수정된 사항을 하나의 브랜치로 병합합니다. 병합하고자 하는 브랜치는 같은 로컬 저장소에 있어야 합니다.

과거에는 모든 병합 작업을 수동으로 해서 어려웠지만, 이제는 이러한 수동 병합 작업을 깃이 대신 처리합니다. 하지만 깃이 모든 코드의 병합을 완벽하게 처리할 수는 없습니다. 아무리 좋은 도구라고 해도 자동으로 반영하지 못하는 것들이 있습니다. 이를 충돌이라고 합니다.

8.1.3 병합 방식

깃의 병합은 브랜치를 기반으로 실행합니다. 각 브랜치를 비교하여 자동 병합하는 형태입니다. 따라서 병합하려면 브랜치를 만들어 브랜치 안에서 수정 작업을 해야 합니다. 사실 파일을 병합하는 것은 그렇게 간단한 일이 아닙니다. 어떤 내용을 수정했고, 새로 추가했는지 판별하기가 어렵기 때문입니다. 병합할 때는 상대적인 기준을 판별하는 알고리즘들이 존재합니다. 이 알고리즘들은 기준점과 수정 사항을 병합하는 처리 로직에 따라 다릅니다.

깃은 병합을 위해 두 가지 기본적인 알고리즘 방식을 제공합니다. 깃에서 충돌 없이 병합하려면 이 두 가지 병합 방식의 차이를 알아야 합니다.

- Fast-Forward 병합
- 3-way 병합

두 방식으로 실습하기 전에 미리 별도의 저장소를 생성해 두겠습니다.

```
$ cd 실습폴더


$ mkdir gitstudy08 새 폴더 만들기


$ cd gitstudy08 만든 폴더로 이동




infoh@DESKTOP MINGW64 /e/gitstudy08


$ git init 저장소 초기화


Initialized empty Git repository in E:/gitstudy08/.git/
```

8.2 Fast-Forward 병합

깃의 가장 간단한 브랜치 병합은 Fast-Forward 방식입니다. 영어 표현을 풀어 쓰면 빨리 감기라고 할 수 있습니다. 일반적으로 Fast-Forward 병합 방식은 혼자 개발할 때 사용합니다.

혼자 개발할 때는 브랜치가 생성된 커밋에 따라 순차적으로 분기됩니다. 또 코드 수정도 순차적으로 할 때가 많습니다. 즉, 브랜치가 분기되지만 전체 커밋 그림으로 보면 모든 변경 사항은 순차적으로 진행됩니다. 이러한 순차적 커밋에 맞추어 병합을 처리하는 방법이 Fast-Forward 병합입니다.

8.2.1 브랜치 생성과 수정 작업

실습을 위해 새로운 feature 브랜치를 생성하겠습니다. 그리고 feature 브랜치로 체크아웃합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (main)

$ git branch feature 브랜치 생성

infoh@DESKTOP MINGW64 /e/gitstudy08 (main)

$ git checkout feature 브랜치 이동

Switched to branch 'feature'

infoh@DESKTOP MINGW64 /e/gitstudy08 (feature)
```



브랜치 변경

브랜치를 생성할 때 분기 기준은 main의 최종 커밋 포인터입니다. 포인터를 확인할 수 있는 rev-parse 명령어로 확인해 봅시다. 첫 번째 커밋 f123d5c와 값이 동일합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (feature)

$ git rev-parse feature 브랜치 커밋 확인

f123d5c9fd0c1f8a2f4ec86f63451a179371ae09
```

8.2.2 병합 위치

깃의 merge 명령어는 브랜치를 병합합니다. merge 명령어는 현재 브랜치를 기준으로 다른 브랜치의 모든 커밋을 병합합니다.

$ git merge 브랜치이름

브랜치를 병합하려면 기준과 대상이 있어야 합니다. 기준은 체크아웃된 현재 브랜치입니다. 따라서 병합하려면 먼저 기준이 되는 브랜치로 이동해야 합니다. 우리는 main 브랜치에서 feature 브랜치로 분기하여 작업했습니다. 작업한 feature 브랜치를 다시 main 브랜치로 병합할 것입니다. 병합을 하려면 먼저 main 브랜치로 체크아웃해야 합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (feature)


$ git checkout main 기준 브랜치로 이동

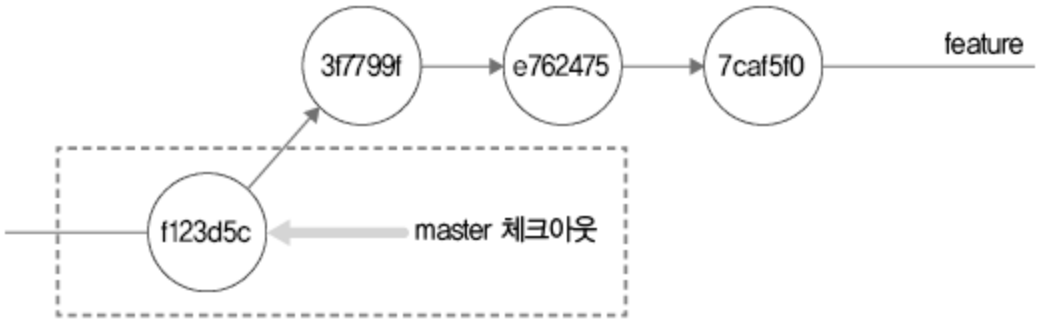

Switched to branch 'main'



infoh@DESKTOP MINGW64 /e/gitstudy08 (main)
```

병합을 할 수 있게 기준 브랜치(main 브랜치)로 이동했습니다.

▼ 그림 8-9 병합을 할 수 있게 기준 브랜치로 이동



8.2.3 Fast-Forward 병합 적용

커밋 작업은 분기된 feature 브랜치에서 모두 수행했습니다. 아직 main 브랜치에는 추가된 커밋이 없습니다. 이러한 상태에서 두 브랜치를 병합합시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (main)


$ git merge feature feature 브랜치 병합


Updating f123d5c..7caf5f0


Fast-forward 병합 방식 표시


 index.htm | 6 ++++++


 1 file changed, 6 insertions(+)
```

feature 브랜치의 커밋을 main 브랜치에 병합했습니다. 병합 메시지를 확인해 볼까요? 메시지에 Fast-Forward 방식을 사용하여 병합했다고 출력됩니다.

이 과정을 그림으로 나타내면 다음과 같습니다.

▼ 그림 8-10 병합한 후 커밋 구조



8.3 3-way 병합

이번에는 깃의 또 다른 병합 알고리즘인 3-way 방식을 알아보겠습니다. 3-way 병합은 좀 더 복잡한 병합을 처리할 수 있는 방법입니다. 여러 개발자와 협업으로 작업하는 경우 대부분 3-way 병합을 사용합니다.

8.3.1 브랜치 생성과 수정 작업

직접 실습하며 3-way 병합을 익히겠습니다. 새로운 작업을 할 hotfix 브랜치를 생성하고, hotfix 브랜치로 체크아웃합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (main)


$ git checkout -b hotfix 브랜치 생성


Switched to a new branch 'hotfix'



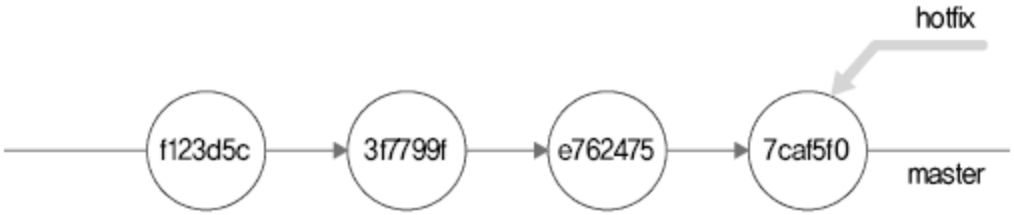

infoh@DESKTOP MINGW64 /e/gitstudy08 (hotfix)
```



이동

현재 브랜치 위치는 hotfix입니다.

▼ 그림 8-12 hotfix 브랜치 생성



8.3.2 main(마스터) 변경

이번에는 브랜치 모양을 변경해 보겠습니다. main 브랜치에도 새로운 커밋을 추가하여 실습을 진행하겠습니다. 먼저 hotfix 브랜치에서 main 브랜치로 이동합니다.

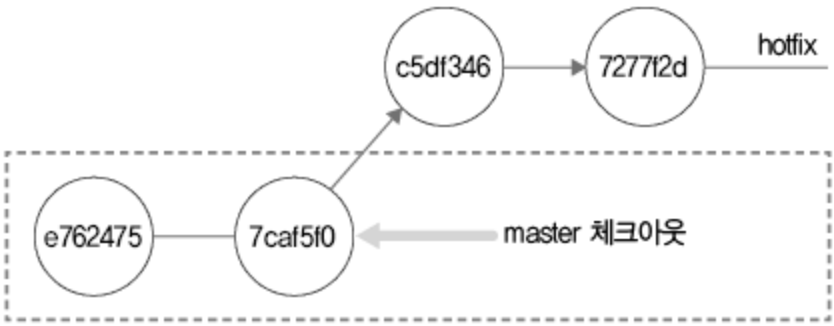
```
infoh@DESKTOP MINGW64 /e/gitstudy08 (hotfix)


$ git checkout main 브랜치 이동


Switched to branch 'main'
```

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (main) 원본 브랜치로 이동
```

▼ 그림 8-15 원본 브랜치로 이동



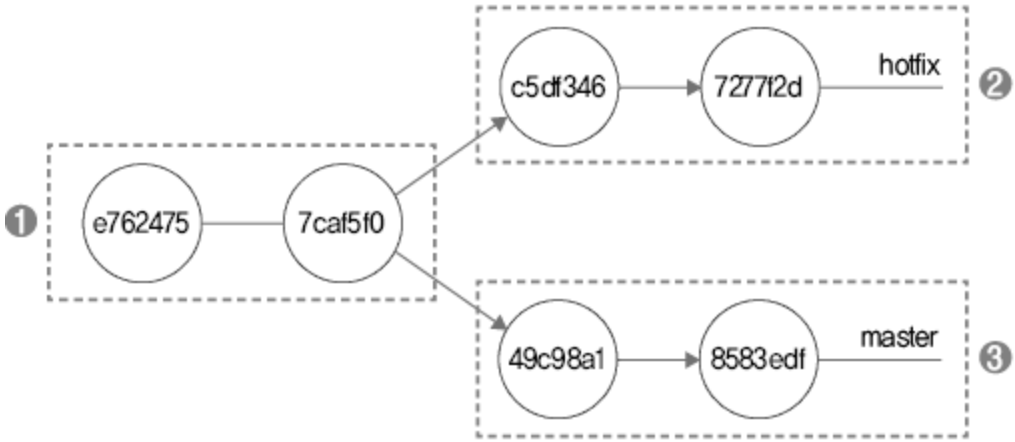
hotfix 브랜치의 마지막 커밋은 7277f2d입니다. 그리고 main 브랜치의 마지막 커밋은 7caf5f0입니다. 시간적으로 좀 더 앞 단계인 main 브랜치에 새로운 커밋을 추가해 보겠습니다.

8.3.3 공통 조상

앞의 실습에서 브랜치별로 각각 커밋하면 두 브랜치로 갈라지는 모습을 보았습니다. 이처럼 브랜치 모양이 갈라지는 형태로 나뉠 때는 Fast-Forward 방식의 알고리즘을 적용하여 병합할 수 없습니다. 이때는 다른 병합 알고리즘인 3-way 방식을 이용해야 합니다.

두 브랜치를 병합하려면 먼저 분할 기준인 공통 커밋을 찾아야 합니다. 이를 공통 조상 커밋이라고 합니다. 공통 조상 커밋을 포함하는 브랜치와 새로운 두 브랜치, 이렇게 3개를 하나로 병합해야 합니다. 브랜치가 3개 있다고 해서 3-way 병합이라고 합니다.

▼ 그림 8-18 브랜치 3개 병합



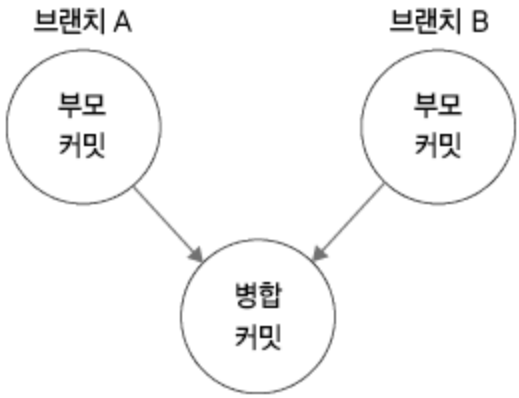
깃은 3-way 병합을 할 때 공통 조상 커밋을 자동으로 찾아 줍니다. 이는 다른 VCS들보다 서로 다른 브랜치를 편리하게 병합할 수 있는 깃의 장점입니다.

8.3.4 병합 커밋

병합은 각 브랜치에서 독립적으로 작업된 소스를 파일 하나로 결합합니다. 하지만 브랜치별로 각각 작업한 내용을 병합하는 과정은 힘들고 어렵습니다. 하지만 깃을 사용하면 복잡한 병합도 좀 더 손쉽게 처리할 수 있습니다.

3-way 병합은 두 브랜치에서 공통 조상 커밋을 자동으로 찾아 주며, 공통 조상 커밋을 기준으로 브랜치를 병합합니다. 그리고 병합을 성공적으로 완료한 후에는 새로운 커밋을 추가로 하나 생성합니다. 새로 생성된 커밋을 병합 커밋이라고 합니다. 병합 커밋은 부모 커밋이 2개라는 특징이 있습니다.

▼ 그림 8-19 병합 커밋



그럼 앞에서 실습한 hotfix 브랜치를 main 브랜치에 병합해 봅시다. hotfix 브랜치를 병합하려면 먼저 기준이 되는 main 브랜치로 체크아웃해야 합니다. 다른 브랜치에 있다면 git checkout main로 체크아웃하세요.

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (main) 기준 브랜치


$ git merge hotfix 기준 브랜치에 hotfix 병합


Auto-merging index.htm


Merge made by the 'recursive' strategy. 3-way 병합


 index.htm | 3 +++


 1 file changed, 3 insertions(+)
```

8.3.5 병합 메시지

3-way 병합을 간단히 실습해 보았습니다. 앞에서 3-way 병합을 할 때 새로운 병합 커밋이 생성되었습니다. 3-way 병합은 Fast-Forward 병합과 달리 병합 메시지가 필요합니다. 그리고 “Merge branch ‘hotfix’”라고 커밋 메시지가 자동 삽입된 것을 확인할 수 있습니다. 깃은 두 브랜치를 병합한 후에 새로운 커밋을 하면서 동시에 메시지를 자동 생성합니다.

자동으로 작성되는 메시지 외에 직접 커밋 메시지를 작성할 수도 있습니다. merge 명령어를 실행할 때 -e 또는 --edit 옵션을 사용하면 됩니다.

$ git merge 브랜치이름 --edit

그럼 직접 병합 메시지를 작성해 봅시다. 실습을 위해 바로 앞에서 진행한 병합을 취소하고 메시지를 추가하여 다시 병합하겠습니다. reset 명령어로 바로 앞의 병합을 취소할 수 있습니다. 리셋은 9장에서 자세히 설명하겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (main)


$ git reset --hard HEAD^ 병합 취소


HEAD is now at 8583edf add menu4




infoh@DESKTOP MINGW64 /e/gitstudy08 (main)


$ git merge hotfix --edit 병합 명령
```

8.4 브랜치 삭제

브랜치를 병합한 후에는 병합한 브랜치를 어떻게 관리할지 결정해야 합니다. 일반적으로 병합한 이후에는 병합된 브랜치를 삭제합니다. 하지만 지속적인 통합과 개발을 해야 하는 브랜치라면 병합 후에도 계속 남겨 둡니다.

Note 깃 플로

깃 플로(git flow)는 브랜치 관리 기법입니다. 수많은 브랜치 작업을 규격화해서 브랜치를 쉽게 다룰 수 있도록 하는 전략이라고 생각하면 됩니다. 깃 플로에는 기본적으로 main, feature, develop, release, hotfix 브랜치가 있습니다. 이 중에서 develop 브랜치는 main 브랜치에 병합한 후에도 삭제하지 않고 계속 유지합니다. 이처럼 오랫동안 유지하는 브랜치를 long-running 브랜치라고 합니다.

8.4.1 병합 후 삭제

병합된 브랜치의 커밋은 모두 원본 브랜치에 적용됩니다. 따라서 중복되는 커밋을 가지는 별도의 브랜치를 유지할 필요는 없습니다. 불필요한 브랜치는 삭제합니다.

$ git branch -d 브랜치이름

브랜치를 삭제할 때는 -d 옵션을 사용합니다. 참고로 -d 옵션은 병합을 완료한 브랜치만 삭제할 수 있습니다. 따라서 실습에서 성공적으로 병합한 feature 브랜치는 -d 옵션으로 삭제할 수 있습니다. main 브랜치에 병합된 hotfix 브랜치를 삭제하겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (main)


$ git branch -d hotfix 브랜치 삭제


Deleted branch hotfix (was 7277f2d).
```

병합을 완료하지 않은 브랜치를 삭제하고 싶다면 대문자 -D 옵션을 사용해야 합니다.[1]

8.5 충돌

실습을 그대로 따라 했다면 모든 병합을 성공적으로 처리했을 것입니다. 하지만 실제 개발 환경에서는 실습과 달리 다양한 문제가 발생합니다. 대표적으로 여러 사람과 개발한 코드 일부분이 충돌(complicit)하는 경우입니다. 보통 충돌은 3-way 병합이 실패한 경우입니다.

8.5.1 충돌이 생기는 상황

여러 사람과 개발 작업을 하다 보면 예상외로 충돌이 자주 발생합니다. 대부분의 충돌 원인은 같은 위치의 코드를 동시에 수정했기 때문입니다. 파일을 수정할 때 여러 개발자가 서로 다른 위치를 수정했다면 깃에서 서로 다른 위치의 소스를 자동으로 병합하기 때문에 문제가 없습니다. 하지만 파일에서 동일한 위치에 두 명 이상이 서로 다르게 수정했다면 충돌이 발생합니다.

즉, 같은 위치를 동시에 수정하면 두 수정 중 어떤 것이 맞는지 깃에서 자동으로 알 수 없기 때문에 충돌이 발생합니다. 이때 깃은 충돌 오류라고 알려 주고, 개발자에게 직접 수정하여 충돌을 해결하라고 요청합니다. 이러한 병합 충돌은 상당히 자주 발생합니다.

8.5.2 실습을 위한 충돌 만들기

실습을 위해 인위적인 충돌 상황을 만들어 보겠습니다. 새로운 footer 브랜치를 만들고 체크아웃합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (main) 기준 브랜치


$ git checkout -b footer 기준에서 새로운 브랜치 파생


Switched to a new branch 'footer'
```

독립된 footer 브랜치에서 index.htm 파일의 <footer>~</footer> 부분 코드를 수정하고 커밋하겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (footer)


$ code index.htm VS Code 실행
```

index.htm

```
<!DOCTYPE html>

<html>

<head>

    <meta charset="utf-8" />

    <meta name="viewport" content="width=device-width, initial-scale=1">

    <title>Page Title</title>

</head>

<body>

    <header>

        <ul>

            <li>깃 소개</li>

            <li>깃 설치</li>

        </ul>

    </header>
```

```
    <h1>hello GIT world!</h1>

    <footer>

    copyright all right 2018 reserved

             by hojinlee

    </footer>

</body>

</html>
```

8.5.3 수동으로 충돌 해결

병합 충돌이 발생하면 결국 수동으로 해결해야 합니다. 직접 소스 코드를 보고 충돌된 부분을 확인한 후 코드를 수정합니다.

먼저 충돌한 소스 코드를 확인합니다. 깃은 충돌이 발생하면 충돌된 코드 내용을 기호와 함께 표시합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (main|MERGING)
```

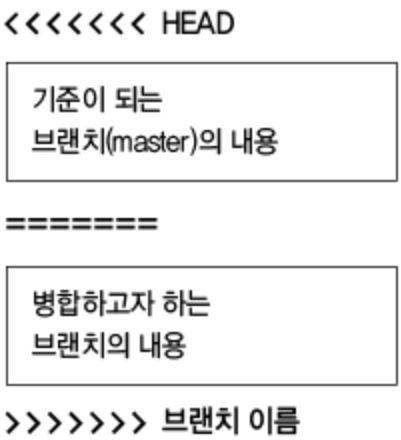
```
$ code index.htm VS Code 실행
```

▼ 그림 8-27 index.htm 파일의 충돌 사항 표시



충돌은 두 부분으로 표시됩니다. 하나는 기준이 되는 브랜치 내용이고, 다른 하나는 병합하고자 하는 브랜치 내용입니다.

▼ 그림 8-28 충돌 코드

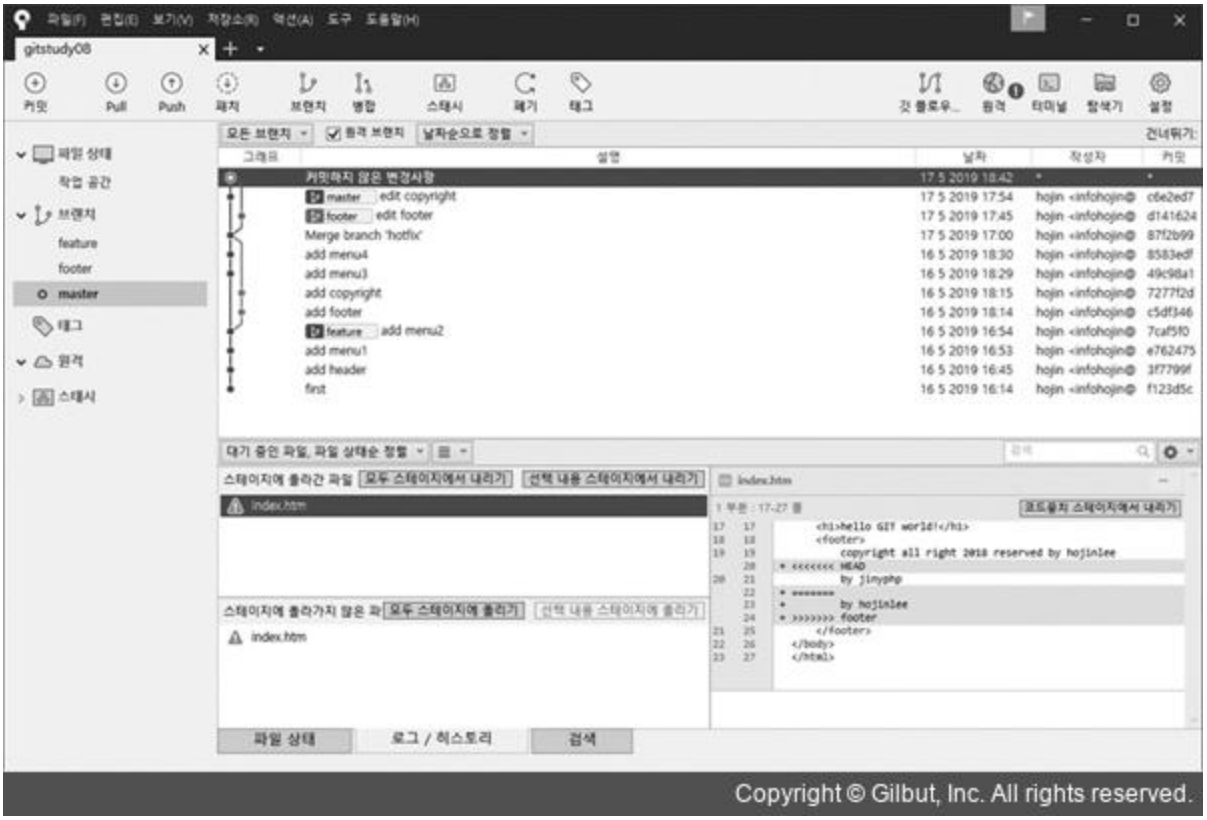


8.5.4 소스트리에서 충돌 해결

소스트리에서도 병합 충돌을 확인할 수 있습니다. 이미 8.5.3절에서 충돌을 해결했기 때문에 8.5.2절까지 진행하여 실습을 위한 충돌을 만든 상황이라고 가정한 채 설명하겠습니다.

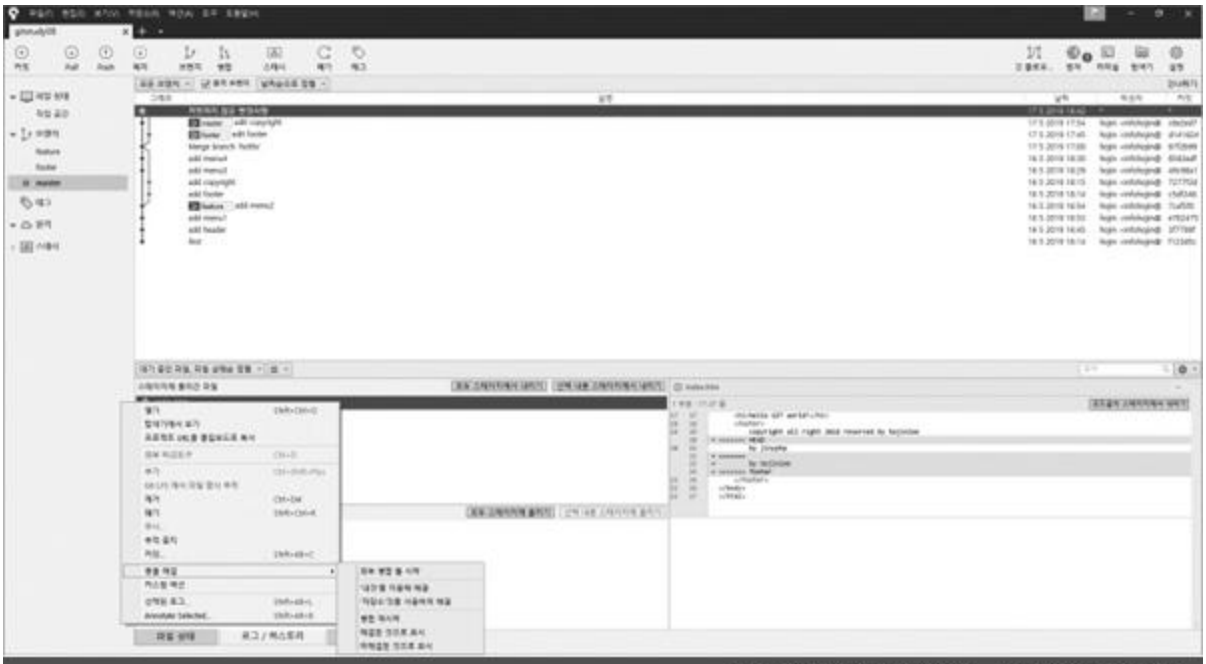
충돌이 발생하면 '스테이지에 올라간 파일'과 '스테이지에 올라가지 않은 파일' 두 영역에 파일이 표시됩니다. 파일을 클릭하면 충돌 메시지를 확인할 수 있습니다.

▼ 그림 8-29 소스트리에서 충돌 확인



소스트리는 병합 커밋을 자동으로 생성할 수 있는 옵션을 제공합니다. 충돌한 파일에서 마우스 오른쪽 버튼을 누릅니다.

▼ 그림 8-30 소스트리에서 충돌 해결



Copyright © Gilbut, Inc. All rights reserved.

8.6 브랜치 병합 여부 확인

다수의 브랜치가 있을 때는 어느 브랜치가 병합을 완료한 것인지 알기 어렵습니다. 브랜치를 병합한 후에 바로 병합된 브랜치를 삭제한다면 이러한 혼동을 줄일 수 있을 것입니다.

깃은 병합한 브랜치와 병합하지 않은 브랜치를 구분하는 옵션을 제공합니다.

$ git branch --merged

main 브랜치에서 이 명령어를 실행해 봅시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (main)


$ git branch --merged 브랜치 목록


  feature


  footer


* main
```

병합한 브랜치는 별표(*) 기호로 표시됩니다. 병합을 완료한 브랜치는 앞에서 배운 -d 옵션을 사용하여 삭제할 수 있습니다.

병합하지 않은 브랜치는 --no-merged 옵션으로 확인할 수 있습니다.

$ git branch --no-merged

병합하지 않은 브랜치는 -d 옵션으로 삭제되지 않으므로 -D 옵션을 사용합니다.

.

8.7 리베이스

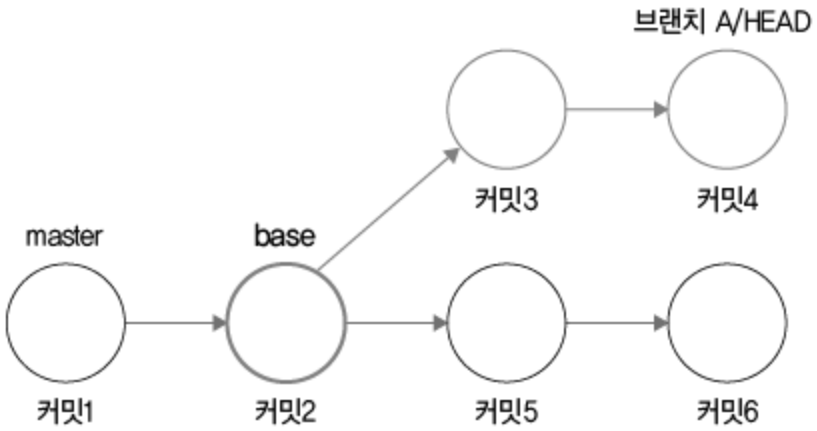
브랜치를 합치는 방법은 두 가지입니다. 앞에서 배운 병합(merge)과 이 절에서 학습할 리베이스(rebase)입니다. 이번에는 커밋 순서를 재배열하는 리베이스 병합을 알아보겠습니다.

리베이스는 커밋의 트리 구조를 재배열합니다. 커밋을 재배열하는 변경 결과가 병합과 유사합니다. 사실 실무에서는 merge 명령어보다는 커밋을 재배열하는 리베이스를 더 선호하는 편입니다.

8.7.1 베이스

다시 브랜치 개념을 떠올려 봅시다. 모든 브랜치는 뿌리가 있습니다(main 브랜치는 예외입니다). 브랜치는 특정 커밋을 가리키는 포인터입니다. 그리고 가리키는 특정 커밋은 브랜치가 파생된 기준이 됩니다. 즉, 브랜치는 커밋 하나를 기준으로 새로운 작업을 진행할 수 있는 분리된 작업 경로를 의미합니다.

▼ 그림 8-33 브랜치의 기준 커밋



그림에서 새로운 브랜치가 커밋2에서 파생됩니다. 새로운 브랜치가 파생되는 커밋2를 베이스(base)라고 합니다. 병합에서는 이를 공통 조상 커밋이라고 합니다.

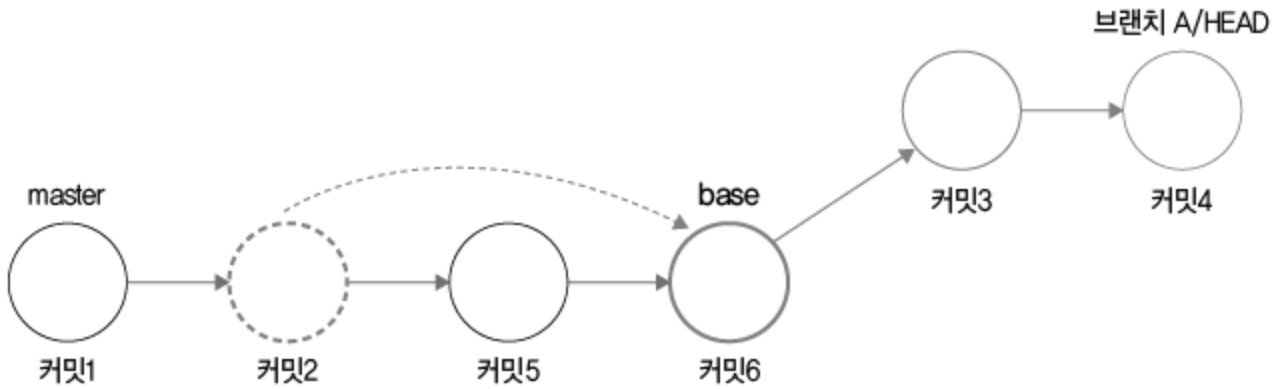
8.7.2 베이스 변경

리베이스(rebase)는 베이스 앞에 '다시'를 의미하는 re가 붙은 단어입니다. 파생된 브랜치의 기준이 되는 베이스 커밋을 변경하는 것입니다.

그럼 브랜치의 베이스는 왜 변경할까요? 커밋의 진행 모습을 단순화하기 위해서입니다. 브랜치가 많아지면 커밋을 관리하고 파악하기 어렵습니다. 마치 꼬여 있는 기찻길처럼 단계별로 커밋을 따라가면서 코드를 확인해야 합니다. 복잡한 브랜치의 수많은 경로는 코드의 진행 상황을 한눈에 파악하기 어렵게 합니다.

리베이스는 코드의 베이스 분기점을 변경하여 마치 하나의 기찻길처럼 만듭니다. 여러 갈래로 갈라지지 않아 커밋의 진행 사항을 좀 더 쉽게 파악할 수 있습니다.

▼ 그림 8-34 베이스 이동

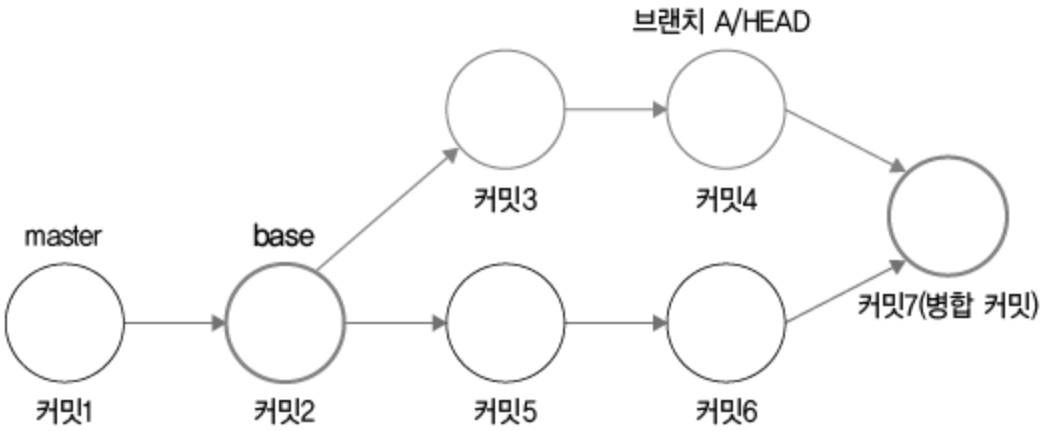


그림처럼 리베이스는 브랜치 A의 공통된 조상인 커밋2를 main 브랜치의 마지막 커밋6으로 변경합니다. 그리고 모든 브랜치의 커밋들을 리베이스된 커밋6 이후로 재정렬합니다.

8.7.3 리베이스 vs 병합

병합은 파생된 두 브랜치를 하나로 합치는 과정입니다. 병합하려면 두 브랜치의 공통 조상 커밋을 먼저 찾아야 합니다. 공통 조상 커밋을 찾으면 서로 다르게 커밋이 진행된 두 브랜치를 3-way 방식으로 병합할 수 있습니다. 공통 조상 커밋은 두 브랜치를 병합하는 베이스 커밋입니다. 병합하는 두 브랜치는 순차적으로 커밋을 비교하면서 마지막 최종 커밋을 생성합니다.

▼ 그림 8-35 3-way 병합



반면에 리베이스는 두 브랜치를 서로 비교하지 않고 순차적으로 커밋 병합을 시도합니다. 리베이스에서 브랜치의 커밋을 결합하는 순서를 살펴봅시다.

8.7.4 리베이스 명령어

리베이스 작업은 rebase 명령어를 사용합니다. 또는 다른 명령어의 옵션으로 리베이스 기능을 실행할 수 있습니다.

$ git rebase 브랜치

실습을 위해 index.htm 파일을 수정하고 커밋하겠습니다. 먼저 리베이스 실습을 위한 새로운 description 브랜치를 생성합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (main)


$ git checkout -b description 브랜치 생성


그리고 description 브랜치에서 index.htm 파일을 수정하고 커밋합니다.


infoh@DESKTOP MINGW64 /e/gitstudy08 (description)


$ code index.htm VS Code 실행
```

index.htm

```
<!DOCTYPE html>

<html>

<head>

    <meta charset="utf-8" />

    <meta name="viewport" content="width=device-width, initial-scale=1">

    <title>Page Title</title>

</head>

<body>

    <header>

        <ul>

            <li>깃 소개</li>

            <li>깃 설치</li>

            <li>커밋</li>

            <li>브랜치</li>
```

```
        </ul>

    </header>

    <h1>hello GIT world!</h1>

    <h2>깃은 소스의 변경 이력을 관리할 수 있습니다.</h2>

    <footer>

            copyright all right 2020 reserved

            by jiny

    </footer>

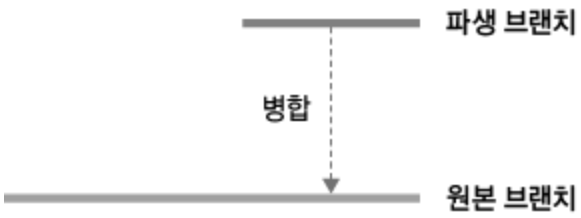
</body>

</html>
```

8.7.5 리베이스 병합

리베이스는 병합 기준 브랜치가 merge 명령어와 반대입니다. merge 명령어를 사용한 병합은 현재의 기준 브랜치에서 다른 브랜치를 읽어 와서 결합합니다.

▼ 그림 8-38 merge 명령어를 사용한 병합



하지만 리베이스는 병합되는 브랜치 방향이 반대입니다.

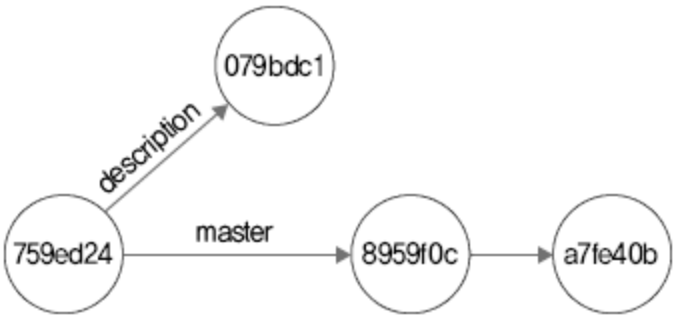
▼ 그림 8-39 리베이스 병합



대부분 처음에는 이러한 병합 기준을 종종 혼동하곤 합니다.

지금까지 실습 과정을 그림으로 나타내면 다음과 같습니다.

▼ 그림 8-40 description 브랜치의 작업 내역



8.7.6 리베이스되었는지 확인

리베이스는 베이스 커밋을 변경하여 병합한다고 했습니다. 베이스 커밋을 변경하는 과정에서 커밋들은 재배치 작업을 합니다. 이 과정에서 커밋의 해시 값이 변경됩니다.

리베이스한 후 로그를 확인해 봅시다. 커밋의 해시 값을 주의 깊게 살펴보세요.

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (description)

$ git log -3 로그 확인

commit 48caea016f0e330cfc1dfcd587996fa9a32042fd (HEAD → description)

Author: hojin <infohojin@gmail.com>

Date:   Sat May 18 17:27:09 2019 +0900

    add description



commit a7fe40bb622f6c8af0b1f25b0d86ea96c7896c50 (main)

Author: hojin <infohojin@gmail.com>

Date:   Sat May 18 17:28:59 2019 +0900

    add menu6



commit 8959f0cca024504cfe321adf440c9c888f8e4693

Author: hojin <infohojin@gmail.com>
```

Date: Sat May 18 17:28:29 2019 +0900

    add menu5

8.7.7 리베이스 후 브랜치

소스트리에서 리베이스를 완료한 후 브랜치를 다시 살펴봅시다. 리베이스 병합 이후 커밋은 정리되었지만, 브랜치 모양이 약간 다릅니다.

▼ 그림 8-43 리베이스 후 브랜치의 마지막 커밋 위치



일반적으로 병합을 한 후 두 브랜치는 같은 커밋 ID를 가리킵니다. 하지만 소스트리에서 확인된 브랜치 위치는 서로 다릅니다.

▼ 그림 8-44 브랜치의 HEAD 포인터



8.7.8 리베이스 충돌과 해결

리베이스는 기준점을 변경합니다. 리베이스 역시 병합 과정에서 충돌이 발생할 수 있습니다. 리베이스 충돌 또한 사용자가 직접 수동으로 해결해야 합니다.

실습을 위해 리베이스 충돌 환경을 만들어 보겠습니다. 먼저 새로운 menu 브랜치를 생성한 후 체크아웃합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (main)


$ git checkout -b menu 브랜치 생성
```

index.htm에서 코드 메뉴를 일부 변경하고 두 번 커밋하겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (menu)


$ code index.htm VS Code 실행
```

index.htm

...

깃 소 개

깃 설 치

커 밋

브 랜 치

병 합

리 베 이 스

...

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (menu)

$ git commit -am "edit menu5" 등록 및 커밋

[menu 8381da6] edit menu5

 1 file changed, 1 insertion(+), 1 deletion(-)




infoh@DESKTOP MINGW64 /e/gitstudy08 (menu)

$ code index.htm VS Code 실행
```

8.7.9 rebase 명령어로 커밋 수정

마지막 커밋은 --amend 옵션으로 수정할 수 있습니다. 이 방법 외에 rebase 명령어로도 최종 커밋을 수정할 수 있습니다.

실제 병합은 아니지만 리베이스는 커밋 위치를 재조정하여 병합과 유사한 효과를 보입니다. 그리고 Fast-Forward 병합을 이용하여 선형 구조 형태로 브랜치 모양을 정리하죠. 리베이스는 커밋을 재조정하는 것 외에도 여러 커밋을 한 커밋으로 묶을 수 있습니다. 이때는 -i 옵션을 사용합니다.

실습하기 전에 현재 저장소 상태를 확인합시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy08 (main)

$ git log -3 로그 확인

commit 690cc959dd410aad516c17c3bd1b46e3e022b2b6 (HEAD → main)

Author: hojin <infohojin@gmail.com>

Date:   Sat May 18 18:08:28 2019 +0900

    edit menu5



commit 93aa6eb8ad3cf50845dd04d9b4e92838279df2b4

Author: hojin <infohojin@gmail.com>

Date:   Sat May 18 18:12:56 2019 +0900

    edit submenu for menu5



commit 48caea016f0e330cfc1dfcd587996fa9a32042fd

Author: hojin <infohojin@gmail.com>
```

Date: Sat May 18 17:27:09 2019 +0900

    add description

8.7.10 리베이스할 때 주의할 점

리베이스는 커밋 위치와 해시 값을 변경합니다. 저장소를 외부에 공개했다면 공개된 순간부터 커밋은 리베이스를 사용하지 않는 것이 원칙입니다.

리베이스는 외부로 코드를 푸시하거나 공개하기 전에 로컬에서만 실행하는 것이 좋습니다. 외부에 공개된 커밋을 리베이스하면 커밋 위치와 해시 값이 변경되어 너무 혼란스럽습니다. 공개된 커밋을 변경할 때는 다음 장에서 배울 revert 명령어를 사용합니다.

8.8 정리

다수의 개발자와 협업할 때 병합과 충돌은 매우 자주 발생합니다. 충돌을 해결하고 병합을 처리하는 것은 쉬운 작업이 아닙니다. 개발 과정에서 병합 충돌을 최소화하고 예방하려면, main 브랜치 내용을 자주 반영하여 병합하는 것이 좋습니다.

원격 저장소의 main 브랜치를 모니터링하고, 변화된 부분을 즉시 반영하면서 작업하면 충돌을 최소화하거나 예방할 수 있습니다.

1 브랜치 삭제 방법은 회사마다 운영 정책에 따라 조금씩 다릅니다.

2 git rebase --continue 명령어를 실행했는데 완벽히 리베이스되지 않았다면, git add 명령어를 한 번 더 실행한 후 git rebase --continue 명령어를 실행해 보세요.